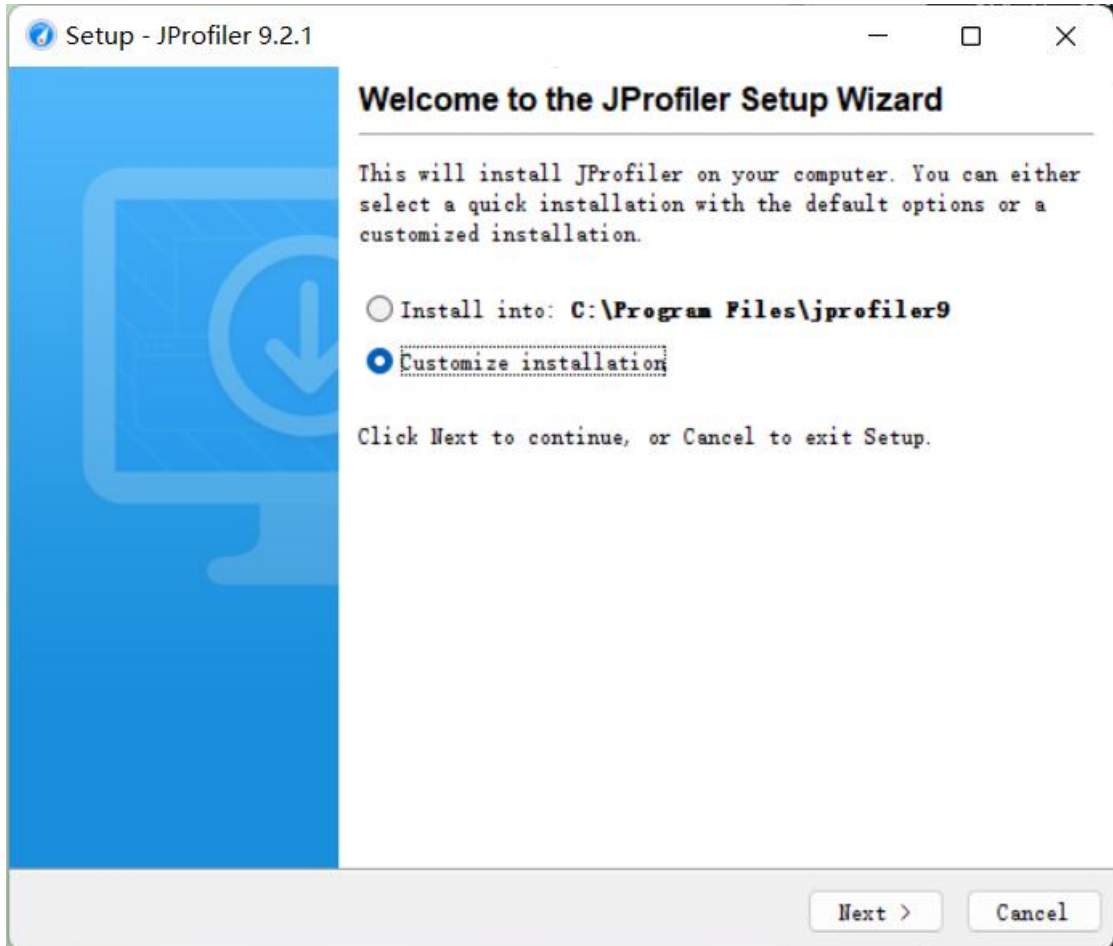
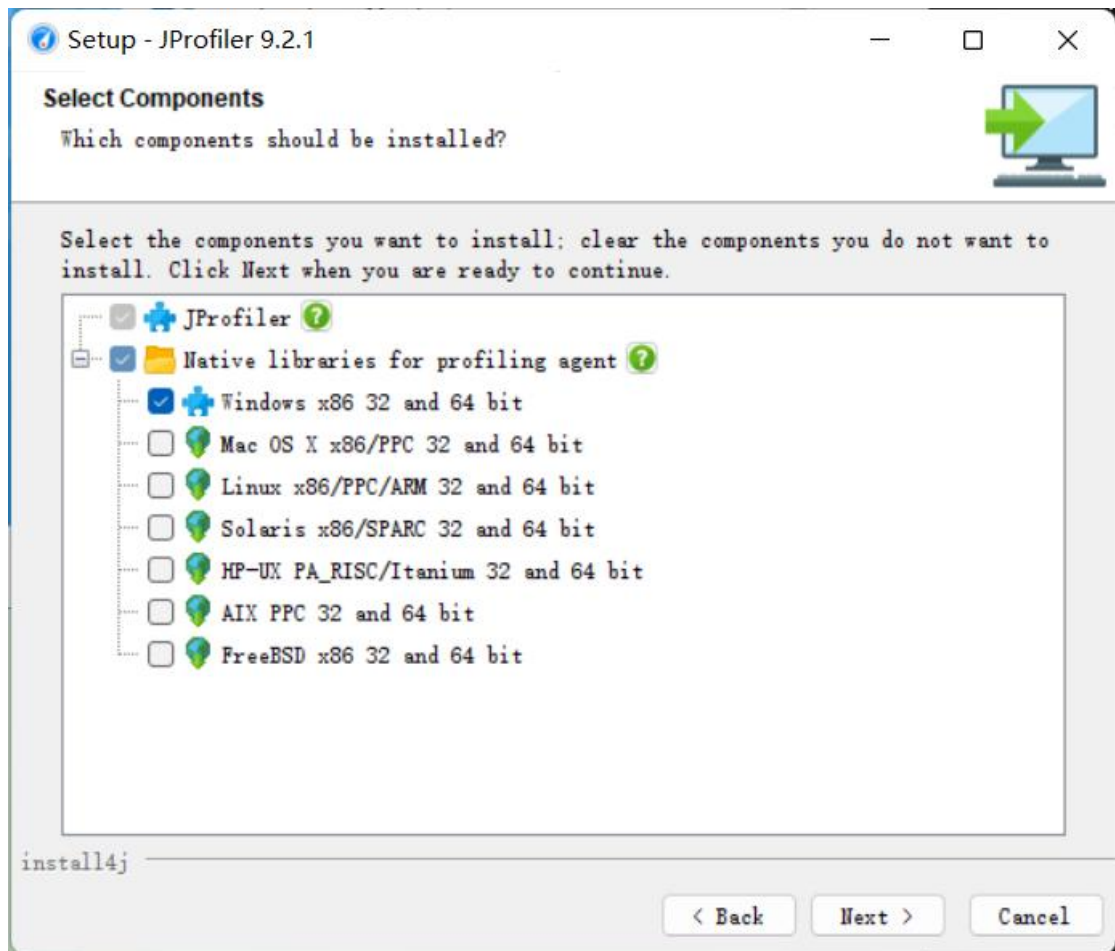
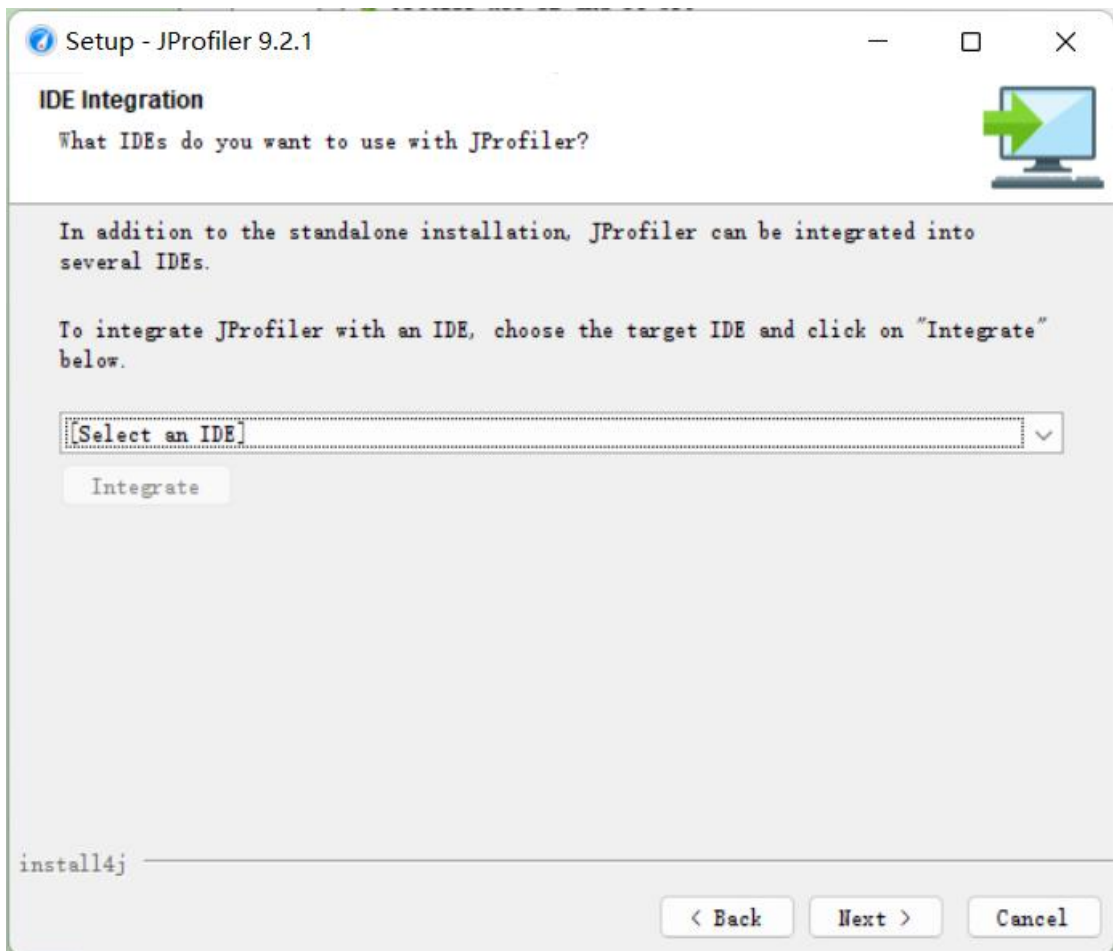


第六次实验

1. Jprofile 安装

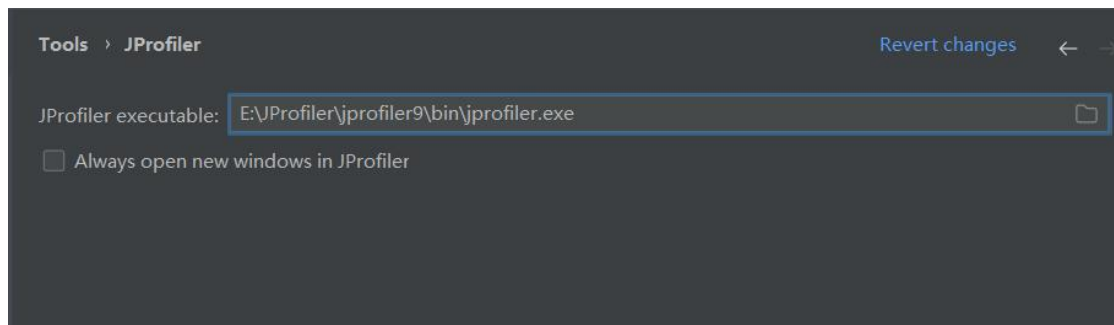
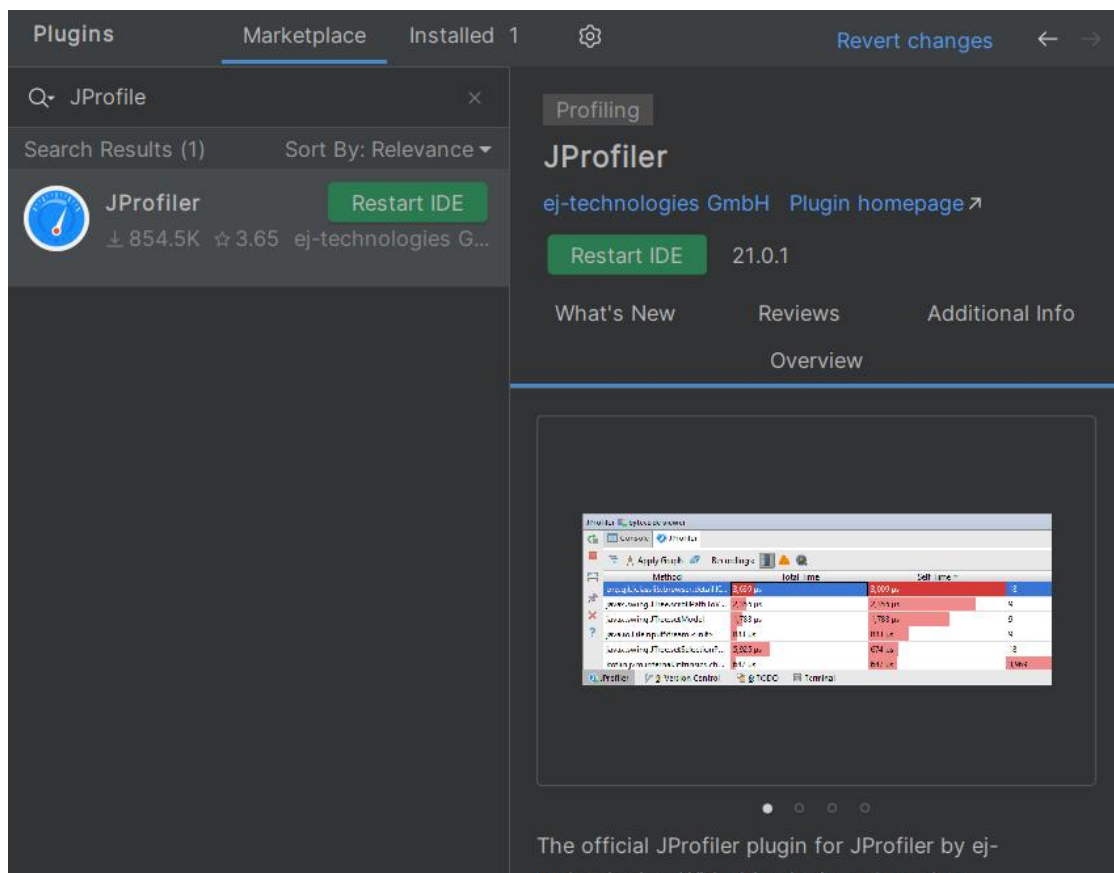






这里旧版本的 JProfiler 不支持 IDEA2024，不进行选择。

2. 初始化配置



3. 内存泄漏监控

自己编写一个纯 java 内存泄漏的程序完成实验，避免使用 test 项目的 tomcat 配置的麻烦。

```
6 1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class MemoryLeakDemo {
5     // 模拟内存泄漏: 静态List持续持有对象引用, GC无法回收
6     private static List<byte[]> leakList = new ArrayList<>(); 2 usages
7
8     public static void main(String[] args) throws InterruptedException {
9         System.out.println("开始模拟内存泄漏...");
10        int i = 0;
11        while (true) {
12            // 每次分配 1MB 数据, 添加到静态列表 (不会被GC回收)
13            byte[] data = new byte[1024 * 1024]; // 1MB
14            leakList.add(data);
15            i++;
16            System.out.println("已分配 " + i + " MB, 总计: " +
17                (leakList.size()) + " 个对象");
18            Thread.sleep(500); // 每500ms分配一次
19        }
20    }
21 }
22 }
```

对 byte[] 数值打标记

Local attach [3] - JProfiler 9.2.1

Session View Profiling Window Help

Start Center Detach Save Session Snapshot Settings Start Recordings Stop Recordings Start Tracing Run GC Add Bookmark Report View Settings Help Freeze View Show In Heap Walker Current

Session Profiling View specific

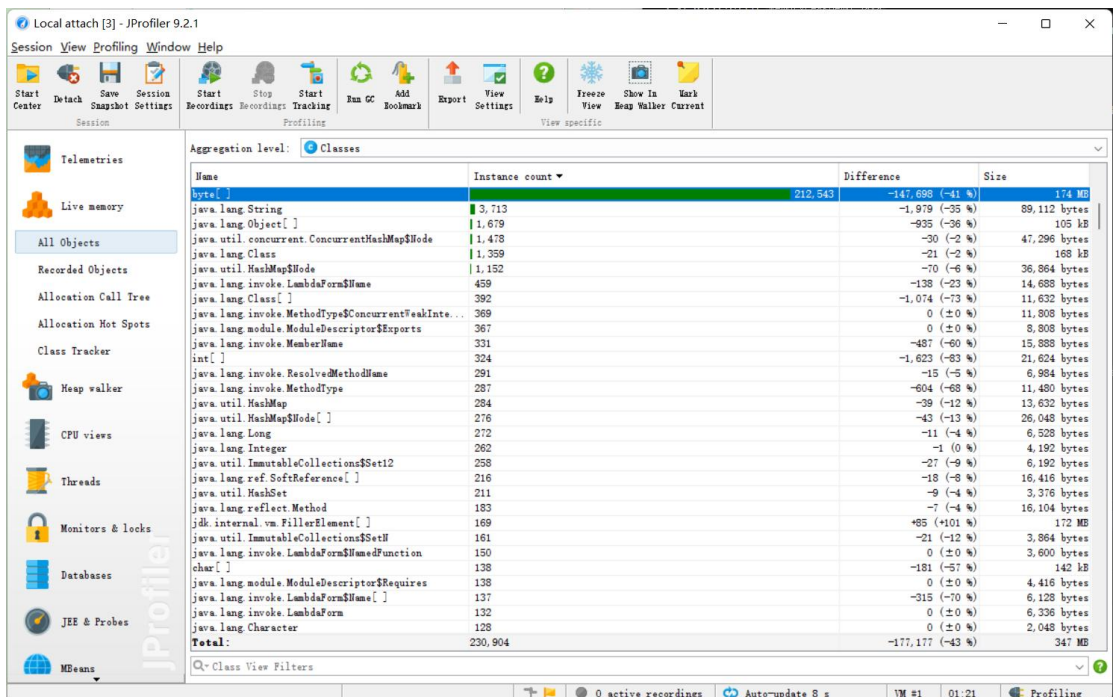
Aggregation level: Classes

Name	Instance count	Difference	Size
byte[]	360,241	0 (±0 %)	59,622 kB
java.lang.Object	6,587	0 (±0 %)	105 kB
java.io.ByteArrayOutputStream	6,454	0 (±0 %)	154 kB
java.io.DataOutputStream	6,454	0 (±0 %)	258 kB
java.lang.String	5,692	0 (±0 %)	136 kB
jdk.internal.org.objectweb.asm.SymbolTable\$Entry	3,142	0 (±0 %)	175 kB
java.lang.Object[]	2,614	0 (±0 %)	147 kB
int[]	1,947	0 (±0 %)	66,224 bytes
java.util.concurrent.ConcurrentHashMap\$Node	1,508	0 (±0 %)	48,256 bytes
java.lang.Class[]	1,466	0 (±0 %)	42,532 bytes
java.lang.Class	1,380	0 (±0 %)	171 kB
java.util.HashMap\$Node	1,222	0 (±0 %)	39,104 bytes
java.lang.invoke.MethodType	891	0 (±0 %)	35,640 bytes
java.lang.invoke.MemberName	818	0 (±0 %)	39,264 bytes
java.lang.StringBuilder	717	0 (±0 %)	17,208 bytes
java.util.ImmutableCollections\$SetN\$SetIterator	613	0 (±0 %)	14,712 bytes
java.lang.invoke.LambdaForm\$Name	597	0 (±0 %)	19,104 bytes
jdk.internal.org.objectweb.asm.ByteVector	480	0 (±0 %)	11,520 bytes
java.lang.invoke.LambdaForm\$Name[]	452	0 (±0 %)	21,512 bytes
java.lang.invoke.MethodType\$ConcurrentFeeInte...	369	0 (±0 %)	11,808 bytes
java.lang.module.ModuleDescriptor\$Exports	367	0 (±0 %)	8,808 bytes
java.util.ImmutableCollections\$Set12\$1	335	0 (±0 %)	8,040 bytes
java.util.KeyValueHolder	324	0 (±0 %)	7,776 bytes
java.util.HashMap	323	0 (±0 %)	15,504 bytes
char[]	319	0 (±0 %)	206 kB
java.util.HashMap\$Node[]	319	0 (±0 %)	30,416 bytes
java.lang.invoke.ResolvedMethodName	306	0 (±0 %)	7,344 bytes
java.util.ImmutableCollections\$Set12	285	0 (±0 %)	6,840 bytes
java.lang.Long	283	0 (±0 %)	6,792 bytes
jdk.internal.org.objectweb.asm.Label	279	0 (±0 %)	15,624 bytes
Total:	415,672	0 (±0 %)	115 MB

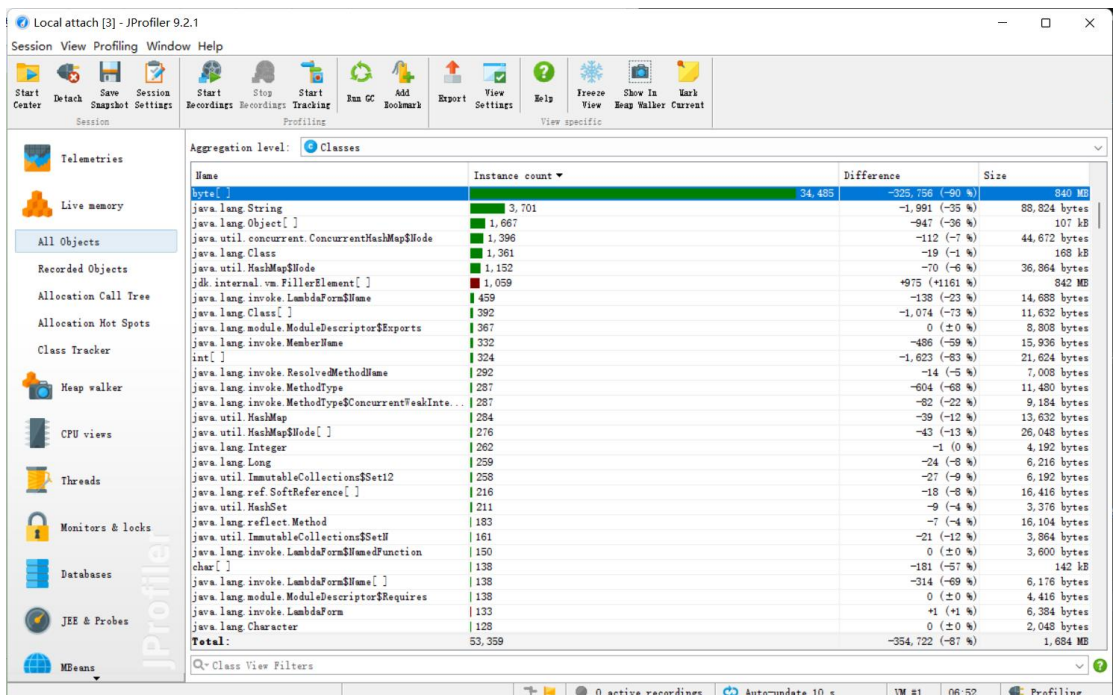
Q~ Class View Filters

0 active recordings Auto-update 8 s VM #1 00:31 Profiling

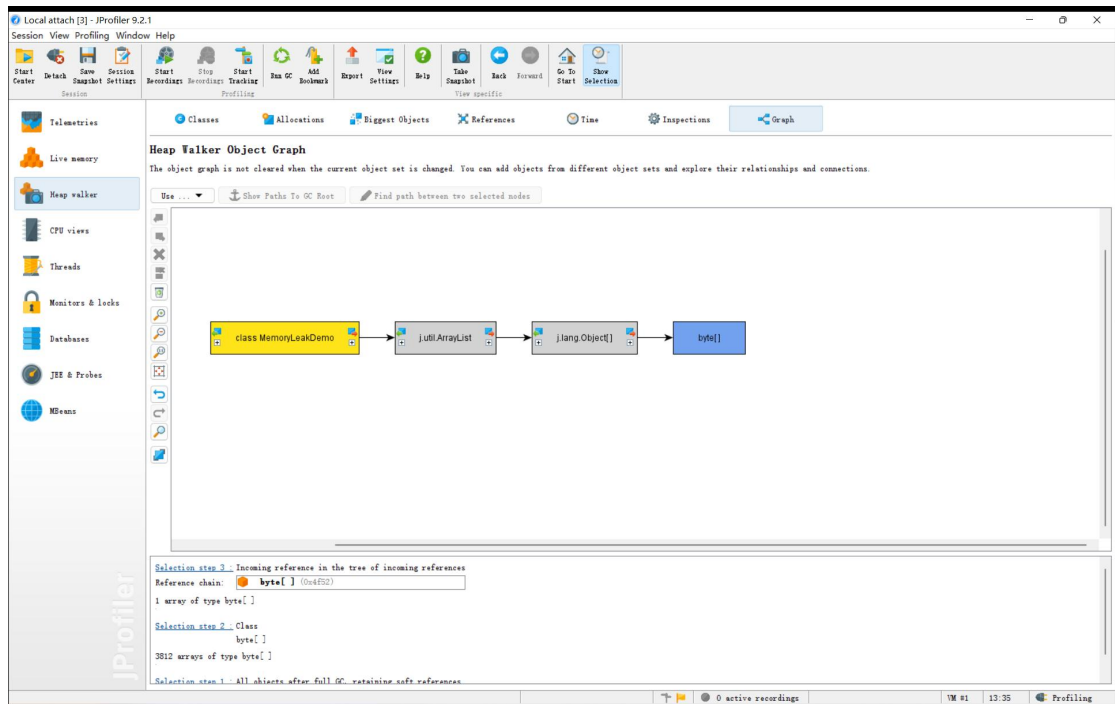
Run GC 后



Difference 列不断增加，说明存在内存泄漏



显示引用链



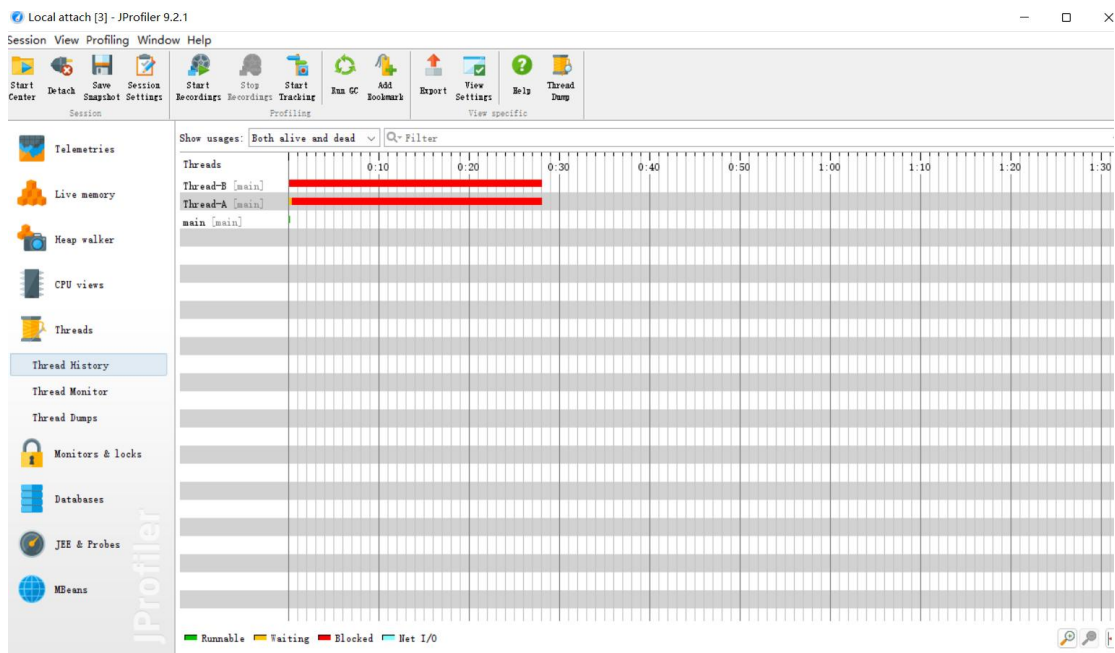
4. 线程阻塞判断


```

1  public class ThreadBlockDemo {
2      private static final Object lock1 = new Object(); 2 usages
3      private static final Object lock2 = new Object(); 2 usages
4
5  public static void main(String[] args) {
6      // 线程A: 先锁lock1, 再锁lock2
7      Thread threadA = new Thread() -> {
8          synchronized (lock1) {
9              System.out.println("线程A获得lock1, 等待lock2...");
10             try { Thread.sleep( millis: 100); } catch (InterruptedException e) {}
11             synchronized (lock2) {...}
14         }
15     }, name: "Thread-A");
16
17     // 线程B: 先锁lock2, 再锁lock1 (与A形成死锁)
18     Thread threadB = new Thread() -> {
19         synchronized (lock2) {
20             System.out.println("线程B获得lock2, 等待lock1...");
21             try { Thread.sleep( millis: 100); } catch (InterruptedException e) {}
22             synchronized (lock1) {
23                 System.out.println("线程B获得lock1");
24             }
25         }
26     }, name: "Thread-B");
27
28     threadA.start();
29     threadB.start();
30 }
31 }
32

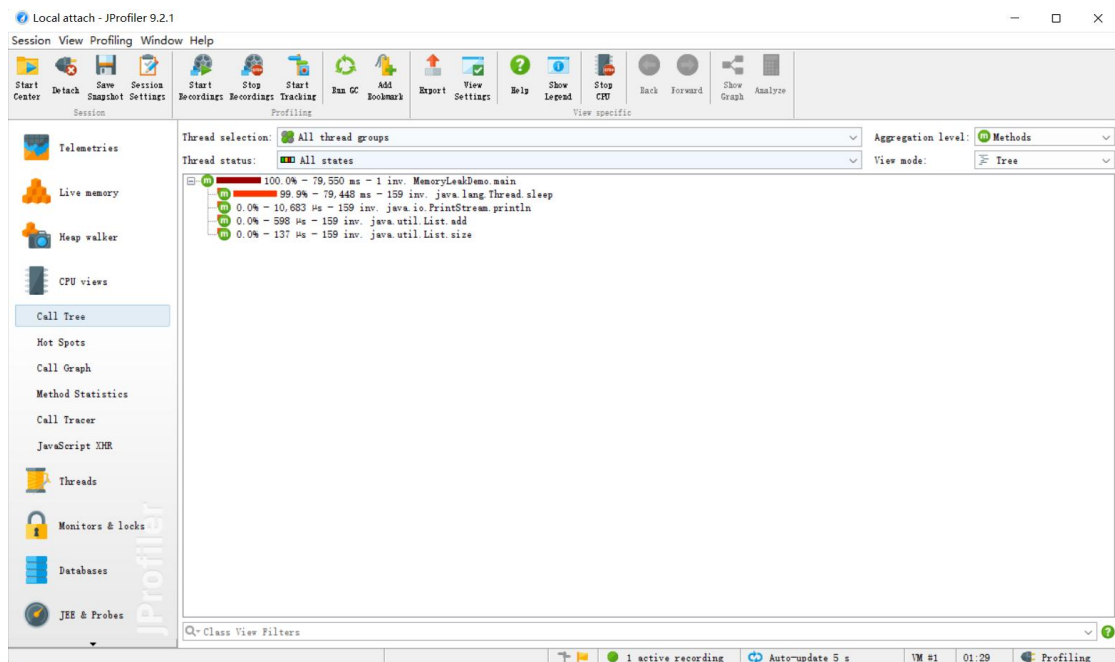
```

Thread-A 和 Thread-B 发生阻塞



5. 运行效能分析

大部分时间都在运行 sleep



Local attach - JProfiler 9.2.1

Session View Profiling Window Help

Start Center Detach Save Snapshot Session Settings Start Recordings Stop Recordings Start Tracking Run GC Add Bookmarks Export View Settings Help Show Legend Stop CPU Show Graph

Session Profiling View specific

Thread selection: All thread groups Aggregation level: Methods

Thread status: All states Filtered classes: Show separately

Hot spot	Self time	Average Time	Invocations
java.lang.Thread.sleep	89,452 ms (99 %)	499 ms	179
MemoryLeakDemo.main	98,148 μs (0 %)	98,148 μs	1
java.io.PrintStream.println	11,971 μs (0 %)	66 μs	179
java.util.List.add	659 μs (0 %)	3 μs	179
java.util.List.size	150 μs (0 %)	0 μs	179

Call Tree

Hot Spots

Call Graph

Method Statistics

Call Tracer

JavaScript XHR

Threads

Monitors & locks

Databases

JEE & Probes

Class View Filters

1 active recording Auto-update 5 s VM #1 01:39 Profiling